

파이썬 활용 게임

우당탕! 건강 헌터즈

2팀
2025.11.19.



Contents

I. 기획 및 시나리오

게임 기획
기획 배경
시나리오

II. 게임 소개 및 주요기능

III. 플로우 차트

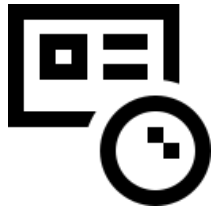


IV. 코딩 및 소스코드

V. 활용방안



게임 기획



게임명



우당탕! 건강 헌터즈
(부제: 칼로리 브레이커)



장르



클릭커 게임



개발환경



Visual Studio
Python 3.12(pygame)



대상층



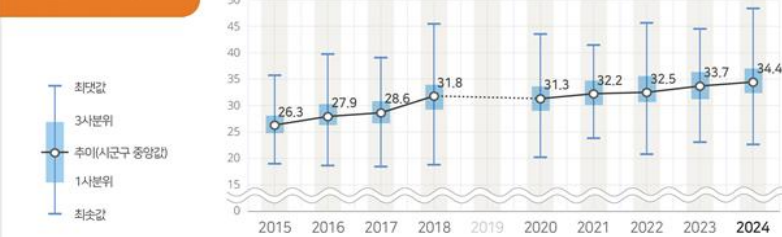
건강관리에
관심 있는 일반인

기획 배경 및 의도

고칼로리 섭취 증가로
인한 비만 문제 심화

- 한국 성인 비만율은 꾸준히 상승(약 34% 수준)
- 고칼로리·고당·고지·가공식품의 과잉 섭취가 핵심 원인

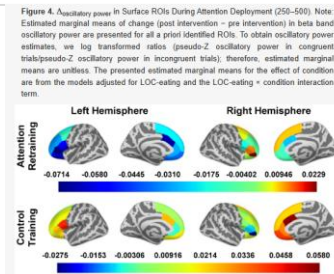
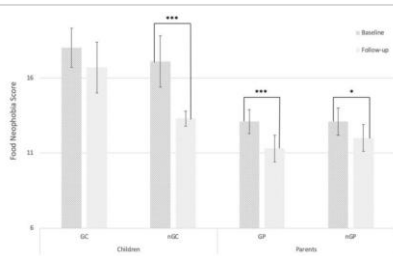
비만율(자가보고)



> 현대인의 식습관 문제
새로운 방식으로 접근

게임 기반 건강
교육(게임화)의 효과성
입증

- 여러 연구에서 고칼로리 회피·건강식 접근 훈련을
게임으로 반복한 참가자들에게서 초기 음식 선택 행동
개선이 관찰됨

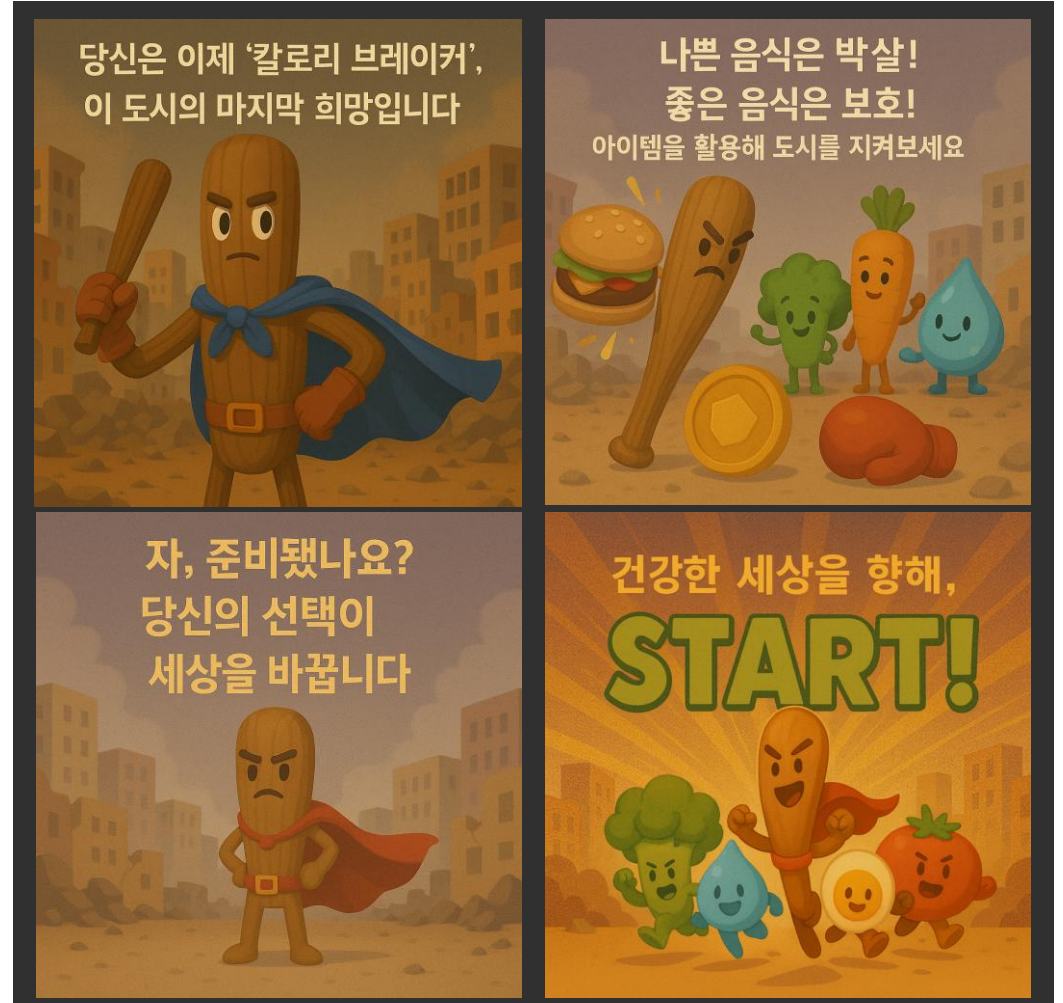


> 행동 변화 촉진의
효과적인 도구



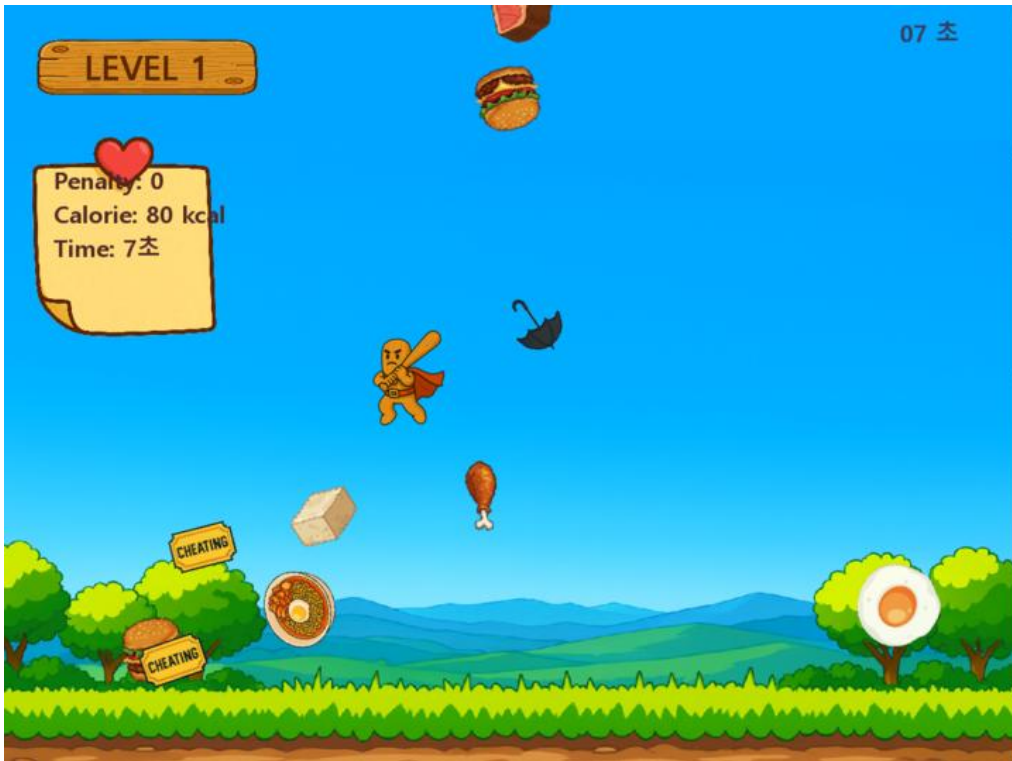
시나리오

현실의 도시가 고칼로리 음식들로 뒤덮이며 건강이 위협받자, 건강 지킴이 '방망이'만이 우리 세계를 구할 수 있다.
'방망이'는 좋은 음식들을 선택해 몸을 회복시키고 나쁜 음식의 공격을 막으며 '성인병 프리' 상태를 달성한다.



게임 소개 및 주요기능

플레이 화면



게임 방법

✓게임 목표:

- 제한된 시간 **30초** 동안
- **총 4,000칼로리** 이상의 고칼로리 음식을 제거하면 성공

✓게임 규칙:

- 고칼로리 음식 클릭 → 제거 → **점수 up!**
- 건강에 좋은 음식 클릭 → 페널티 부과 → **점수 차감**
- 일정 점수 달성 → 난이도 상승 → **속도 증가**
- 필살기 아이템(우산, 망치, 치팅티켓) → 전투력 향상

✓결과 화면:

- 획득 점수 및 성공 및 실패 여부 표시

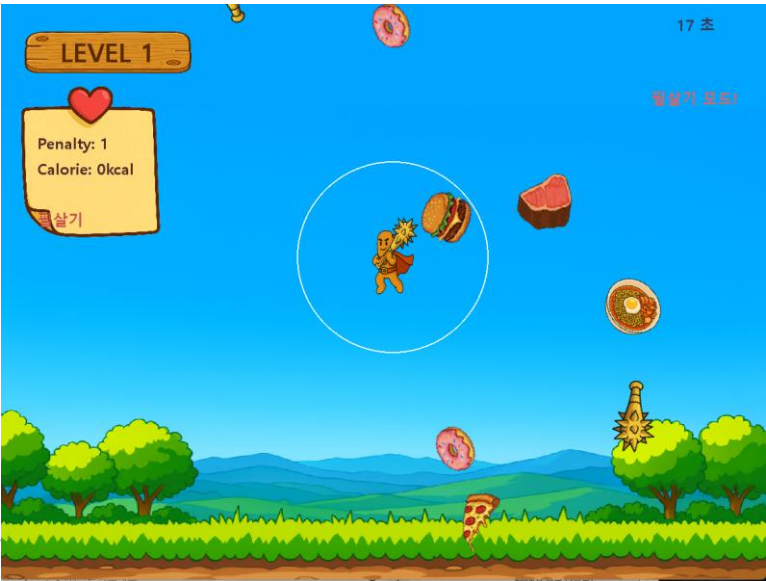
게임 소개 및 주요기능

필살기 (나타나면 클릭!)

	망치
이미지	
기능	타격 범위를 넓혀 한번에 여러 음식을 박살

	우산
이미지	
기능	마을에 보호 우산을 펼쳐 나쁜 음식과 좋은 음식을 자동 필터링

	치팅데이
이미지	
기능	화면 색상이 바뀔 동안 어떤 음식이든 제한없이 박살 가능



게임 소개 및 주요기능

엔딩

성공 화면



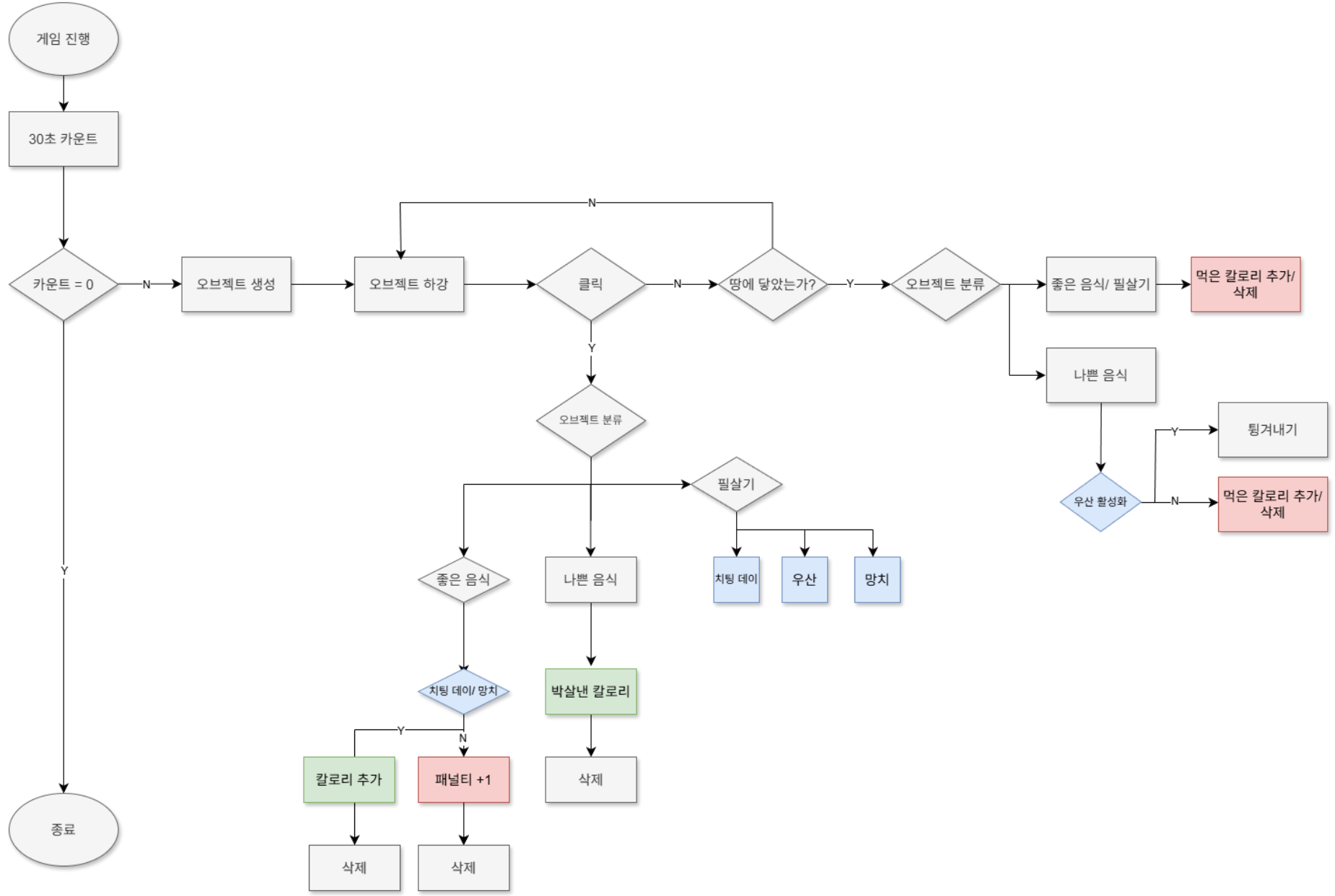
승리!
이웃들의 건강을 지켜냈다!

실패 화면



실패...
이웃들을 나쁜 음식으로부터 지키지 못했다...

플로우차트



소스코드 및 코딩

- Pygame 라이브러리를 통해 메인 루프를 기준으로 게임이 실행되고 종료됨

- 기본 세팅값

```
# ----- 게임 메인 루프 -----
def game_loop():
    foods = [create_food() for _ in range(10)] #음식 10개 생성하며 시작
    explosions = []
    popup_texts = []

    total_bad_cal = 0 # 박살낸 칼로리
    eaten_cal = 0 # 먹은 칼로리
    penalty = 0 # 좋은 음식 실수로 때린 횟수

    level = 1 #레벨업 기준점수
    next_level_score = [0, 200, 400, 600, 900, 1200, 1600,1800,2100,2500]
    level_up_timer = 0

    umbrella_timer = 0
    ultimate_timer = 0
    cheating_timer = 0

    swing_timer = 0

    start_time = pygame.time.get_ticks()
    pygame.mouse.set_visible(False)
    running = True
```

- 루프가 너무 빠르게 돌지 않도록 fps=60으로 지정해 줌

```
while running:
    dt = clock.tick(FPS)
    now = pygame.time.get_ticks()
    remain = max(0, GAME_TIME - (now - start_time))

    if remain <= 0:
        break

    for e in pygame.event.get():
        if e.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        if e.type == pygame.MOUSEBUTTONDOWN and e.button == 1:
            mx, my = pygame.mouse.get_pos()
            swing_timer = 150
```

소스코드 및 코딩

- 일반적인 상황과 필살기 아이템이 사용되었을 때의 상황을 예외처리로 구별함

```
# 일반 모드: 클릭 위치에 맞은 음식/아이템 1개만 처리
for food in reversed(foods):
    if food["rect"].collidepoint(mx, my):
        # 아이템이면 타이머 켜기
        if food["type"] == "item":
            if food["item_type"] == "umbrella":
                umbrella_timer = 4000
            elif food["item_type"] == "ultimate":
                ultimate_timer = 4000
            elif food["item_type"] == "cheatingday":
                cheating_timer = 5000
            explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "bonus"))
            food.update(create_food())
            break

        # 음식일 때
        if food["is_bad"]:
            # 나쁜 음식 맞추면 해당 칼로리만큼 점수
            total_bad_cal += food["cal"]
            explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "bad"))
        else:
            # 좋은 음식인데 치팅데이 중이면 점수 +50
            if cheating_timer > 0:
                total_bad_cal += 50
                explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "bonus"))
            else:
                # 그냥 좋은 음식 때리면 패널티 증가
                penalty += 1
                explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "penalty"))

        # 처리 후 새 음식 생성
        food.update(create_food())
        break
```

```
# 필살기 모드: 광역 타격 + 아이템도 획득
if ultimate_timer > 0:
    for food in foods:
        dx = food["rect"].centerx - mx
        dy = food["rect"].centery - my
        if dx*dx + dy*dy <= (150*150):
            if food["type"] == "item":
                if food["item_type"] == "umbrella":
                    umbrella_timer = 4000
                elif food["item_type"] == "ultimate":
                    ultimate_timer = 4000
                elif food["item_type"] == "cheatingday":
                    cheating_timer = 5000
                explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "bonus"))
                popup_texts.append(PopupText(food["rect"].centerx,
                                         food["rect"].centery, "ITEM!", (255,
                                         200, 0)))
                food.update(create_food())
            else:
                # 필살기 중엔 좋은 음식도 모두 점수
                if food["is_bad"]:
                    total_bad_cal += food["cal"]
                else:
                    total_bad_cal += 50
                explosions.append(Explosion(food["rect"].centerx,
                                         food["rect"].centery, "bonus"))
                food.update(create_food())

    continue
```


소스코드 및 코딩

- 하늘에서 떨어지는 좋은 음식과 나쁜 음식 데이터를 이름, 칼로리, T/F, 색깔, 사진 순서로 구성

- 화면에서 아이템을 움직이기 위해 데이터를 딕셔너리로 구성

- 기타 많은 코드들이 적절한 위치에 이미지를 삽입하거나 그림을 그리기 위해 사용됨
- 기능 구현을 위한 수학적 사용됨

```
# ----- 음식 데이터 -----
FOOD_SPECS = [
    ("라면", 70, True, (255, 120, 120), "라면.png"),
    ("피자", 90, True, (255, 120, 120), "피자.png"),
    ("햄버거", 80, True, (255, 120, 120), "햄버거.png"),
    ("닭다리", 60, True, (255, 120, 120), "닭다리.png"),
    ("도넛", 65, True, (255, 120, 120), "도넛.png"),

    ("계란후라이", 12, False, (120, 255, 120), "계란.png"),
    ("사과", 8, False, (120, 255, 120), "사과.png"),
    ("두부", 9, False, (120, 255, 120), "두부.png"),
    ("생선", 14, False, (120, 255, 120), "생선.png"),
    ("스테이크", 25, False, (120, 255, 120), "스테이크.png"),
]

FOOD_SIZE = 100
```

```
# ----- 음식 / 아이템 생성 -----
def create_item():
    item_type = random.choice(ITEM_TYPES)
    rect = pygame.Rect(
        random.randint(20, WIDTH - 20 - FOOD_SIZE),
        random.randint(-350, -50),
        FOOD_SIZE, FOOD_SIZE
    )
    return {
        "type": "item",
        "item_type": item_type,
        "rect": rect,
        "speed": random.uniform(3.0, 5.8),
        "angle": random.uniform(0, 360),
        "rotation_speed": random.uniform(-3, 3),
        "bounced": False
    }
```

```
# ----- 배트 -----
def draw_bat(surface, mx, my, swing_timer):
    swing = min(1, max(0, swing_timer / 200))
    angle = 1.5 - swing * 2.5

    px = mx + 15 * math.cos(angle)
    py = my + 15 * math.sin(angle)

    handle_len = 30
    barrel_len = 60
    total_len = 130

    def r(px_offset, py_offset):
        px_offset = -px_offset
        return (
            px + px_offset * math.cos(angle) - py_offset * math.sin(angle),
            py + px_offset * math.sin(angle) + py_offset * math.cos(angle),
        )
```

향후 개선 방향

게임 플레이 측면

- ✓ 전체 플레이 시간이 30초로 짧아 레벨 별 난이도 체감 어려움
- ✓ 필살기가 무작위로 난이도와 무관하게 등장



- ✓ 플레이 시간 연장 또는 스테이지 분리를 통한 레벨 별 난이도 구분
- ✓ 필살기 등장 빈도를 레벨 별로 조정

콘텐츠 다양화 측면

- ✓ 테마가 칼로리 한 가지로 칼로리 이외의 건강 관련 정보 학습 아쉬움



- ✓ 칼로리 이외의 영양정보 추가
- ✓ 다양한 건강 이슈 기반의 테마 확장 (고혈압, 당뇨 등)

UX/UI 측면

- ✓ 초반 인트로 게임 방법 설명이 구체적이지 않음



- ✓ 인트로에 게임 방법 설명 구체적으로 안내

활용 방안

건강 교육 분야

- ✓ 식습관 교육·비만 예방 목적의 건강 행동 변화 촉진을 위한 훈련 도구로 활용



건강보험공단/보건소
건강생활 실천 캠페인과 연계

병원·영양상담 클리닉
환자 교육용으로 사용



디지털 헬스/브랜드 커뮤니케이션 분야

- ✓ 게임을 브랜드 메시지 전달, 참여형 이벤트, 헬스 콘텐츠 제작에 활용



모바일 헬스 앱
미니게임 기능으로 내장

브랜드 메시지 전달/
고객 참여형 건강 캠페인

음식 칼로리 학습 및
선택 훈련을 위한
인터랙티브 콘텐츠 제작



자, 준비됐나요?
당신의 선택이
세상을 바꿉니다

감사합니다

